

ECE 759

High Performance Computing for Applications in Engineering

[Fall 2024]

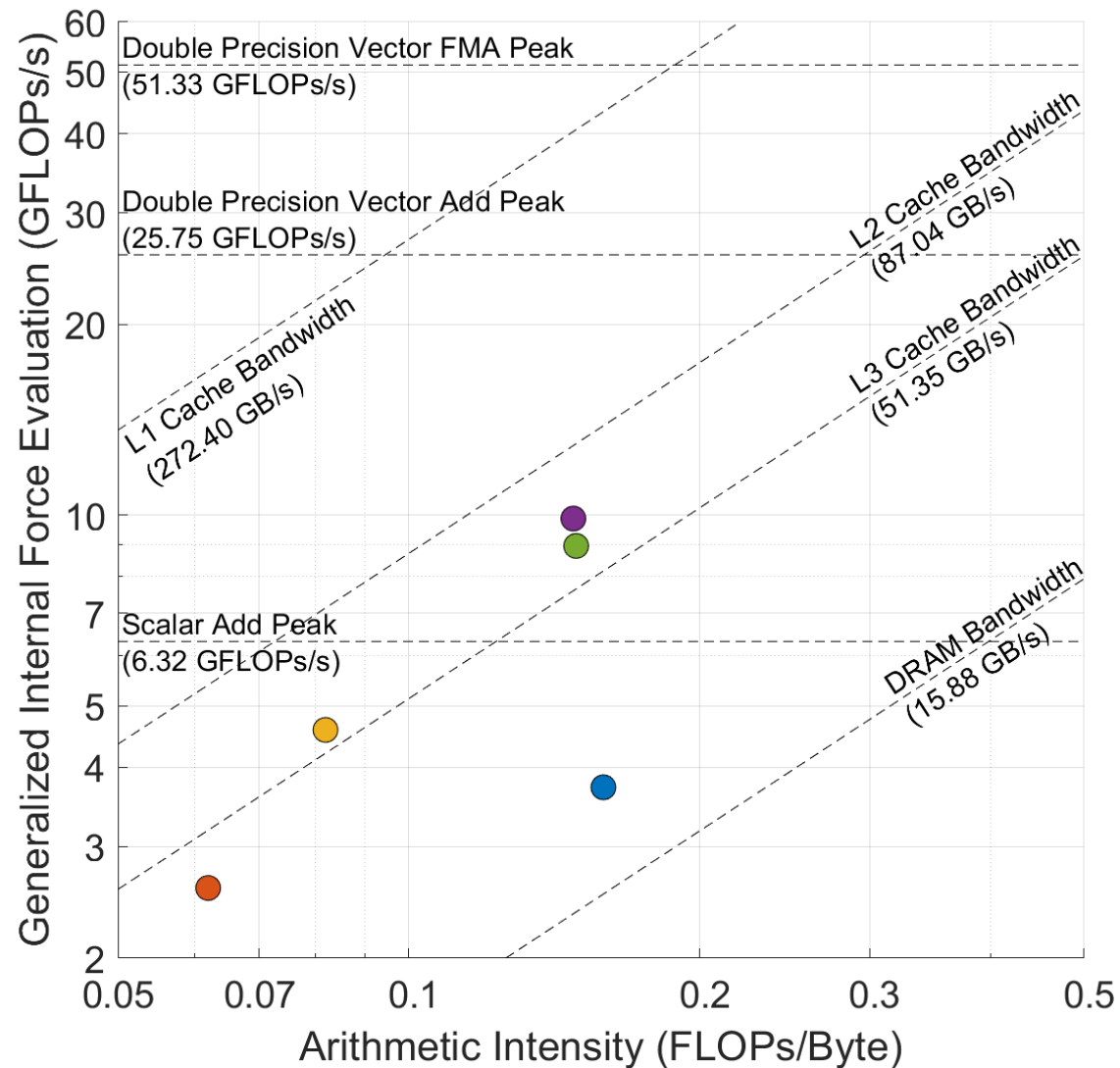
Understand Your Compiler

2024/10/28

Before we get started...

- Quick overview, things discussed last time
 - OpenMP performance issues: cache coherence, false sharing, NUMA effects
- In today's lecture:
 - Code optimization: Roofline analysis, compiler optimization, and SIMD
- Miscellaneous
 - Assignment #4 is released – due on **Wed 10/30 at 23:50 PM**
 - Final project proposal – due on **Friday 11/01 at 23:50 PM**

Roofline Analysis: Visualize the Efficiency of Hardware



- The X coordinate of the point in the plot:
 - Aggregate arithmetic intensity for your code
- The Y coordinate of the point in the plot:
 - Aggregate flops/second rate for your code
- Memory-Bound Region:
 - Low operational intensity; performance is constrained by memory bandwidth
- Compute-Bound Region:
 - High operational intensity; performance is constrained by peak computational throughput.

On having a “high” point for High Performance Computing

- Ideally, we want to achieve both high arithmetic intensity and high FLOPs rate
- Factors that come into play when trying to move the point high up in the roofline plot:
 - a) **Compiler optimization**
 - b) **Memory aspects**: pointers, hierarchy, latencies, bandwidths
 - c) **Instruction level parallelism** (pipelining, jump instructions, branch prediction, wide registers, etc.)
- We'll use a couple of examples to show above come into play

Let's talk next about the **compiler**... bullet a) on previous slide

- When you time/profile your code, you'll have to play with the compile flags
 - Execution speed impacted heavily by compiling flags (3-4X speedup)
- NOTE: Aggressive optimization done by compiler might change behavior of your code
 - Might reorder the instructions to achieve better data locality
 - Might change results due to unintended consequences
 - Might be a consequence of the non-sequential consistency of the memory model

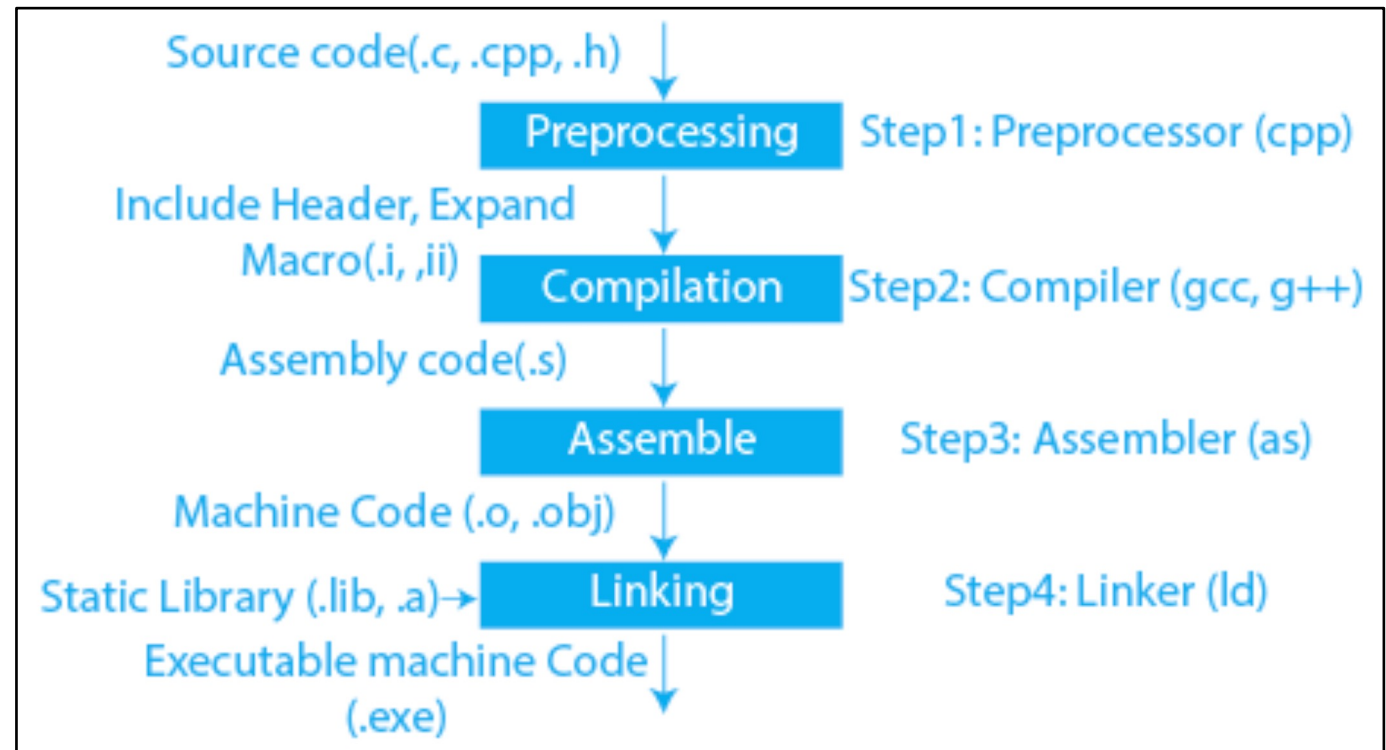
There are compilers and compilers...

- Many compilers out there: gcc, clang, g++, icl, cl, xl, etc.
- A compiler might be available only on one hardware architecture
- A compiler might be available only on one operating system
- Some compilers are more customized than others to the underlying hardware
 - Fair comparison of compilers is not straightforward
 - Different compilers have different flags
 - Different flags may do different things on hardware
 - Different flag combinations may product different results

Compiler	Flags
Gnu	-Ofast -mtune=native -finline-limit=50000
Intel	-O3 -fast -fno-inline-factor -xHost
Cray	-O3 -O ipa5 -h wp
PGI	-acc -ta=tesla,cc35,nordc -Mcuda=maxregcount:80,loadcache:L1

How **compilers** enter the equation

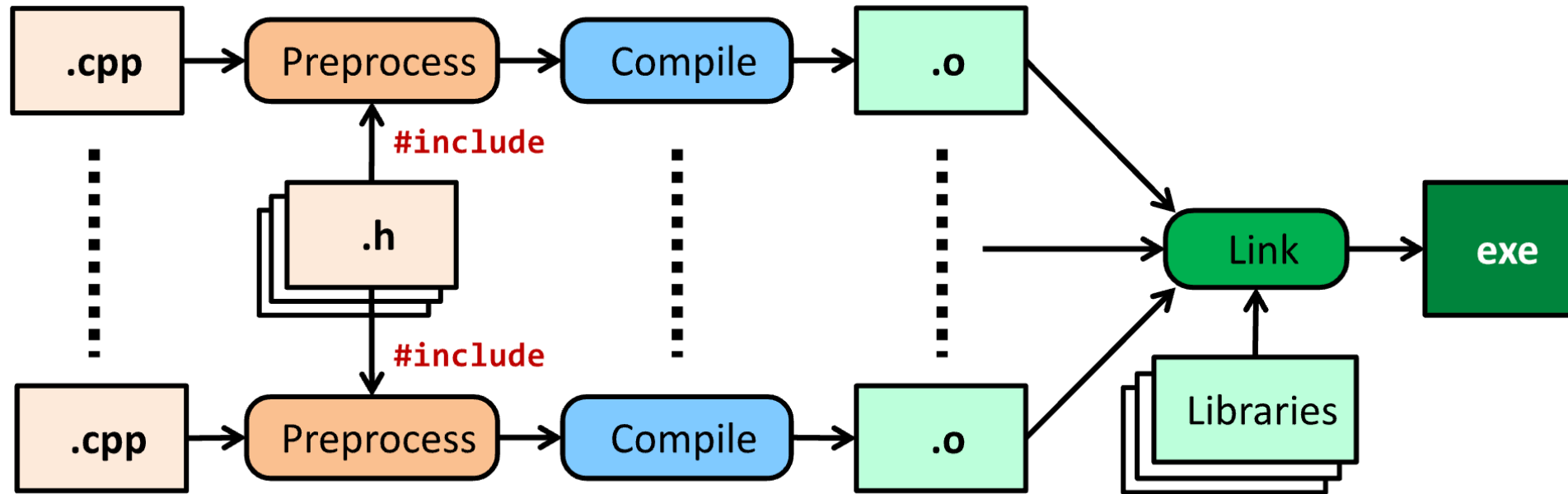
- Compiler expected to provide an efficient mapping of program into the machine instructions
 - IN: your source code
 - OUT: machine instructions (executable)
- A compiler has a tough job
 - Register allocation
 - Instruction selection and ordering
 - Dead code elimination
 - Fix minor inefficiencies in your code
 - ...



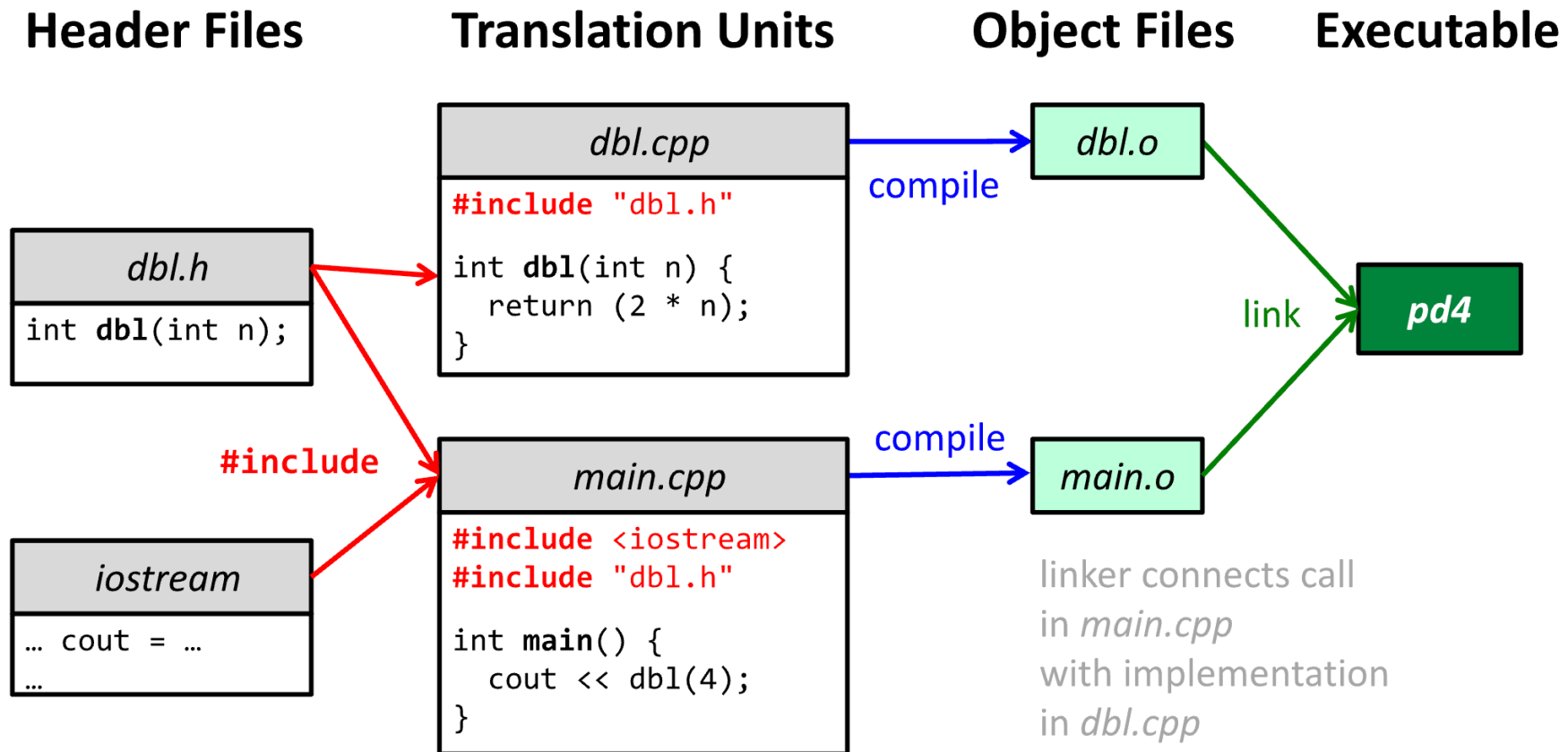
The typical compilation flow of a C++ program

C++ Compilation and Build Model

- **Header** (*.h/*.hpp) and **Translation Units** (TUs) (*.cpp) contain source code
- **Preprocessor** performs text substitutions
- **Compiler** translates TUs into object files
- **Linder** links object files and external libraries into an executable



Example



```
$ g++ -c main.cpp
$ g++ -c dbl.cpp
$ g++ main.o dbl.o -o pd4
```

compiling `main.cpp` yields `main.o`
compiling `dbl.cpp` yields `dbl.o`
linking `main.o` + `dbl.o` yields executable `pd4`

How **compilers** come into play

- Compilers don't improve asymptotic efficiency of your algorithm
 - If your algorithm runs $O(N^2)$, compiler won't reduce it to $O(N)$, for instance
 - Up to programmer to select the right algorithm for the problem and the architecture
 - Basically, you can think of this as engineering-level optimization – *For a given algorithm, how can I optimize the most of its implementation?*
- Compilers have difficulty overcoming “**optimization blockers**”
 - **Data Dependencies:** When operations depend on the results of previous computations, the compiler often has limited flexibility in reordering or parallelizing these instructions
 - **Aliasing:** When two pointers reference the same memory location, it becomes unclear if changes to one pointer will affect the other, limiting the compiler's ability to reorder instructions or optimize memory access
 - **Function Calls:** Calls to external functions (especially with unknown or side effects) act as "black boxes" to the compiler, making it hard to analyze or optimize code around them
 - **Memory Access Patterns:** Irregular or indirect memory access patterns make it challenging for the compiler to optimize memory usage and cache performance

The compiler: has “keyhole vision”

- Most often, compiler ends up doing analysis only at the function level
- Newer versions of gcc and clang do inter-procedural analysis within **individual** files
- **Whole-program analysis** for some reason not done too often
 - Reason: increases substantially amount of time required to generate an executable

```
int functionFromOtherFile(double*);
double someOtherRemoteFunc(double*);

double myFunc() {
    const size_t N = 1024;
    double foo[N];
    double accumulator = 0.;
    for (int i = 0; i < functionFromOtherFile(foo); i++) {
        accumulator += someOtherRemoteFunc(foo);
    }
    return accumulator;
}
```



Compiler in a tough spot: function bodies are often hidden from call sites, compiler can only see their declaration

Compilers, and link time optimization (LTO)

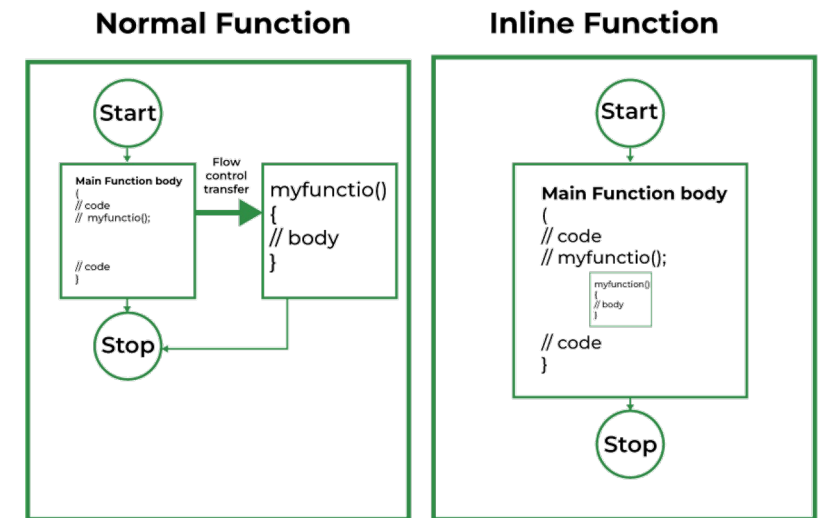
- Compilers are becoming faster and smarter
- It's affordable and profitable to do program-level, not function level optimization
- LTO (also known as LTCG, for “link time code generation”) can be used to allow the compiler to see across translation unit boundaries
 - Note: A translation unit is the fundamental unit of compilation in C++. It's essentially a single source file along with all the header files it directly or indirectly includes, minus any code excluded by preprocessor directives like `#if` or `#ifdef`.
- Likely to provide good speedup for complex codes with many source files

How does LTO work?

- Individual translation units compiled to an **intermediate representation** (IR) instead of machine code
 - An intermediate representation (IR) is a data structure or code used internally by a compiler to represent source code
 - Ex: LLVM IR
- Machine code is generated during the link process—when the entire program is visible
- The compiler can take advantage of this to inline across translation units, or at least use information about the side effects of called functions
- NOTE: Invoking LTO can be done today; controlled through compile flags

Broadening the perspective of the compiler via **inlining**

- Function inlining – very helpful, helps the compiler see more code
 - Wherever you call a function, the compiler simply replaces the call to function foo with the entire body of function foo (the entire function body dropped in)
 - Not always possible, you must explicitly call for it (by function decoration and/or compile flags)
 - Nowadays you can ask the compiler to report back what functions it successfully inlined
- Quick consequences, if you can inline:
 - You have one less jump
 - No stack operations
 - More important: the compiler sees way more code and therefore engages in more fruitful optimizations
 - It's one thing to optimize when you have 5 lines of code, and a different one for 500 lines of code



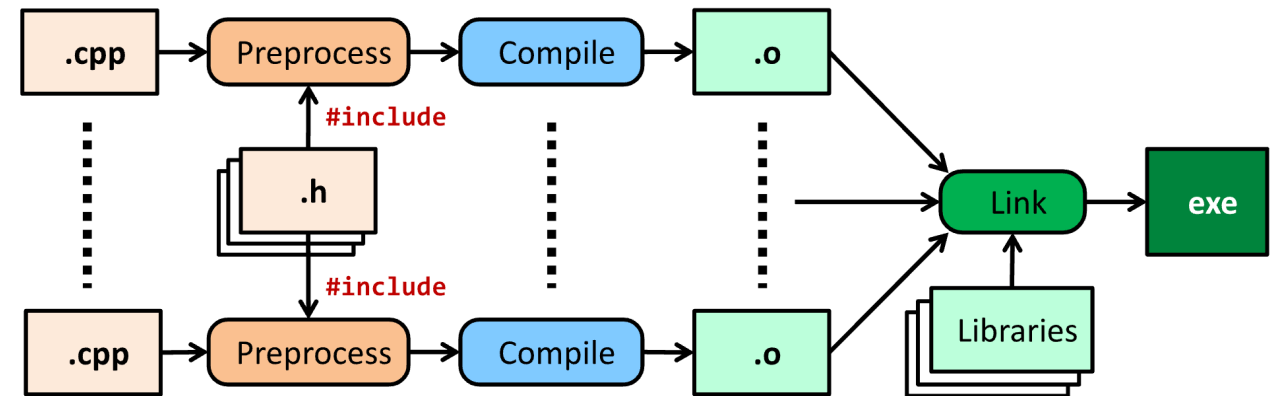
How can you help the compiler?

A) Allow it to see as much code as possible

- Recall that typically compiler sees only one file at a time
 - Can't peek at the body of a function that is defined in a different file

B) Provide flags to convey information to it

- Tell compiler the target architecture
- Tell compiler to unroll loops
- Tell compiler to inline critical functions
- ...

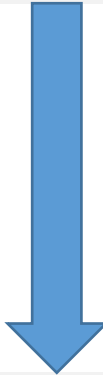


Other tricks pulled off by the compiler [called “strength reduction”]

You wrote this:

```
for (int i = 0; i < 100; ++i)
{
    func(i * 1234);
}
```

This is what the compiler will generate on your behalf
to reduce the costly multiplication operations:



```
for (int iTimes1234 = 0; iTimes1234 < 123400; i += 1234)
{
    func(iTimes1234);
}
```

Other lesser trick: Share Common Subexpressions

- Reuse portions of expressions
- GCC will take care of this for you with `-O1` flag

```
/* Sum neighbors of i,j */
up =    val[(i-1)*n + j];
down =  val[(i+1)*n + j];
left =  val[i*n      + j-1];
right = val[i*n      + j+1];
sum = up + down + left + right;
```

Up
Down
Left
Right

}
Stencil
Operations

Three multiplications: $i*n$, $(i-1)*n$, $(i+1)*n$

```
leaq    1(%rsi), %rax    # i+1
leaq    -1(%rsi), %r8    # i-1
imulq   %rcx, %rsi      # i*n
imulq   %rcx, %rax      # (i+1)*n
imulq   %rcx, %r8      # (i-1)*n
addq    %rdx, %rsi      # i*n+j
addq    %rdx, %rax      # (i+1)*n+j
addq    %rdx, %r8      # (i-1)*n+j
```

```
long inj = i*n + j;
up =    val[inj - n];
down =  val[inj + n];
left =  val[inj - 1];
right = val[inj + 1];
sum = up + down + left + right;
```

One multiplication: $i*n$

```
imulq   %rcx, %rsi      # i*n
addq    %rdx, %rsi      # i*n+j
movq    %rsi, %rax      # i*n+j
subq    %rcx, %rax      # i*n+j-n
leaq    (%rsi,%rcx), %rcx # i*n+j+n
```

Other lesser tricks: Replace costly operations with simpler ones

1. Compiler will use “shift” whenever you multiply by a power of 2
 - $16 * x \rightarrow x \ll 4$
 - Do not do this yourself, let the compiler do it, since it's machine dependent
2. Compiler will go to great lengths to avoid divisions of integers
 - Around 20 times slower than an addition, and 5 times more expensive than multiplication
 - Division by power of 2 is simple (equivalent to shifting bits, compiler does it quickly)
 - Here's a divide by 3, there is no division, actually:

```
unsigned divideByThree(unsigned x){  
    return x / 3;  
}
```

`divideByThree(unsigned int):`



```
    mov     eax, edi           ; eax = edi  
    mov     edi, 2863311531    ; edi = 0xaaaaaaaaab  
    imul    rax, rdi           ; rax = rax * 0xaaaaaaaaab  
    shr     rax, 33            ; rax >>= 33  
    ret
```

Computes 0.33333333337213844 of the input

One common theme for speeding up code: **code motion**

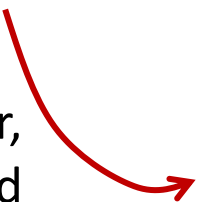
- Code Motion

- Reduce frequency with which some computation is performed
 - Particularly rewarding when a piece of code can get moved out of a loop
 - Typically done by the compiler, under higher optimization levels – if this is possible (more later)
- Caveat: Make sure the new code produces same result (if you, rather than the compiler goes for it)

- Code motion, basic idea, captured in this code snippet:

```
void set_row(double *a, double *b, int i, int n) {  
    for (int j = 0; j < n; j++)  
        a[n*i + j] = b[j];  
}
```

Typically done by compiler,
if no red flags encountered



```
void set_row(double *a, double *b, int i, int n) {  
    int dummy = n * i;  
    for (int j = 0; j < n; j++)  
        a[dummy + j] = b[j];  
}
```

Assembly code, no optimization

```
1 void set_row(double *a, double *b, int i, int n) {  
2     for (int j = 0; j < n; j++)  
3         a[n*i + j] = b[j];  
4 }
```

See line 3 (left) and line 19 (right)

x86-64 gcc 9.2 ☒ Compiler options...

A ☐ Output... ☐ Filter... ☐ Libraries ☐ Add new...

```
1 set_row(double*, double*, int, int):  
2     push    rbp  
3     mov     rbp, rsp  
4     mov     QWORD PTR [rbp-24], rdi  
5     mov     QWORD PTR [rbp-32], rsi  
6     mov     DWORD PTR [rbp-36], edx  
7     mov     DWORD PTR [rbp-40], ecx  
8     mov     DWORD PTR [rbp-4], 0  
9     .L3:  
10    mov     eax, DWORD PTR [rbp-4]  
11    cmp     eax, DWORD PTR [rbp-40]  
12    jge     .L4  
13    mov     eax, DWORD PTR [rbp-4]  
14    cdq     rax  
15    lea     rdx, [0+rax*8]  
16    mov     rax, QWORD PTR [rbp-32]  
17    add     rdx, rax  
18    mov     eax, DWORD PTR [rbp-40]  
19    imul    eax, DWORD PTR [rbp-36]  
20    mov     ecx, eax  
21    mov     eax, DWORD PTR [rbp-4]  
22    add     eax, ecx  
23    cdq     rax  
24    lea     rcx, [0+rax*8]  
25    mov     rax, QWORD PTR [rbp-24]  
26    add     rax, rcx  
27    movsd   xmm0, QWORD PTR [rdx]  
28    movsd   QWORD PTR [rax], xmm0  
29    add     DWORD PTR [rbp-4], 1  
30    jmp     .L3  
31    .L4:  
32    nop  
33    pop     rbp  
34    ret
```



Assembly code, optimization -O1

Multiplication moved out of loop by compiler!

```
1 void set_row(double *a, double *b, int i, int n) {  
2     for (int j = 0; j < n; j++)  
3         a[n*i + j] = b[j];  
4 }
```

Take home message:
the compiler is your friend

```
x86-64 gcc 9.2 (Editor #1, Compiler #1) C++ x  
x86-64 gcc 9.2 -O1  
A Output... Filter... Libraries + Add new...  
1 set_row(double*, double*, int, int):  
2     test    ecx, ecx  
3     jle     .L1  
4     lea     r8d, [rcx-1]  
5     imul    ecx, edx  
6     movsx   rcx, ecx  
7     lea     rcx, [rdi+rcx*8]  
8     mov     eax, 0  
9     .L3:  
10    movsd   xmm0, QWORD PTR [rsi+rax*8]  
11    movsd   QWORD PTR [rcx+rax*8], xmm0  
12    mov     rdx, rax  
13    add     rax, 1  
14    cmp     rdx, r8  
15    jne     .L3  
16    .L1:  
17    ret
```

Bring b[j] into xmm0
Move xmm0 into a
Move value of j into rdx
Increment rax; i.e., j, by 1
Compare j with n (in r8)
Jump to .L3 if not equal

Function calls are optimization blockers

- Procedure to convert UPPER case characters in a string to lower case characters
 - `strlen` is C function that takes an input string pointer and returns its length

```
void lowerV1(char *s)
{
    size_t i;
    for (i = 0; i < strlen(s); i++)
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] -= ('A' - 'a');
}
```

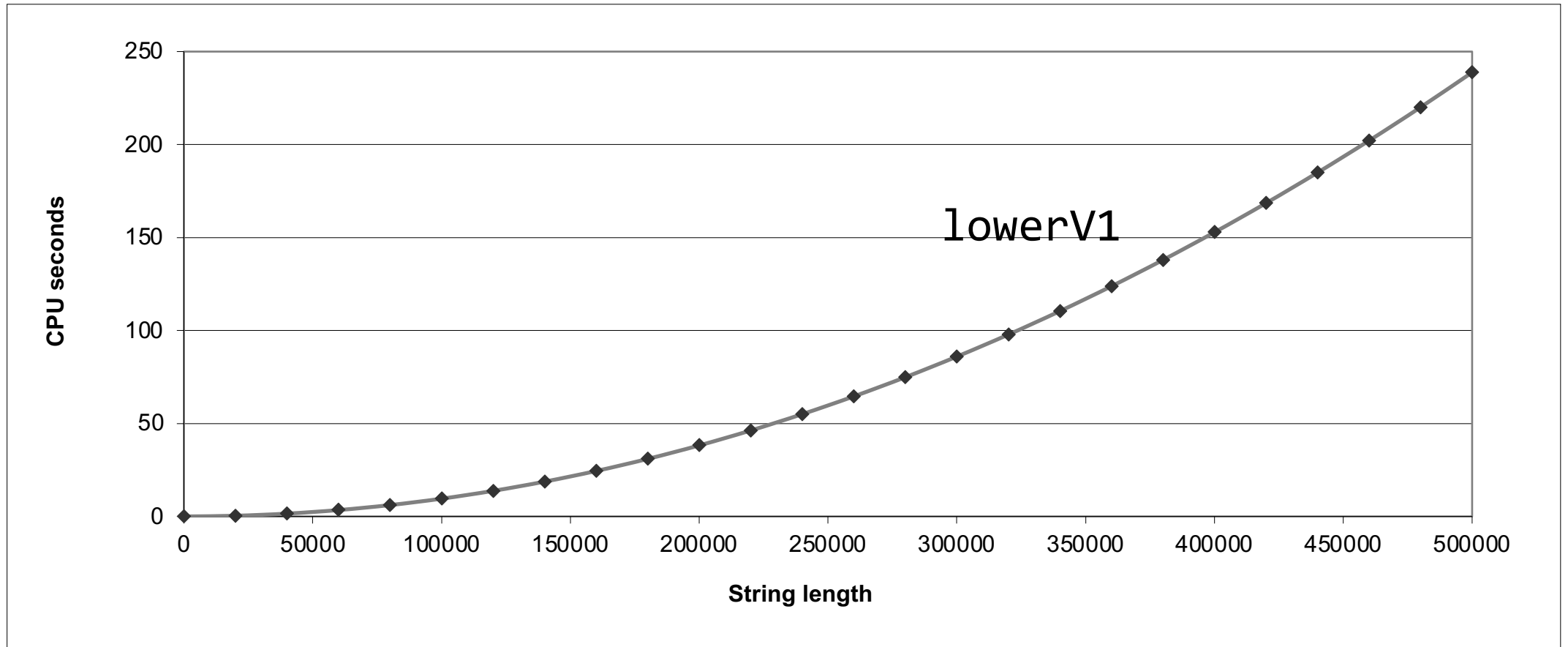
Calling strlen

- strlen performance
 - Only way to determine length of string is to scan its entire length, looking for null '\0' character
- We have N calls to strlen
 - Each call runs loop over N elements to find '\0'
- What's the impact?
 - Overall, this will lead to $O(N^2)$ performance of our solution for conversion to lower case

```
/* My version of strlen */
size_t strlen(const char *s)
{
    size_t length = 0;
    while (*s != '\0') {
        s++;
        length++;
    }
    return length;
}
```

Lower Case Conversion Performance: lowerV1

- Time quadruples when double string length – *Quadratic performance*



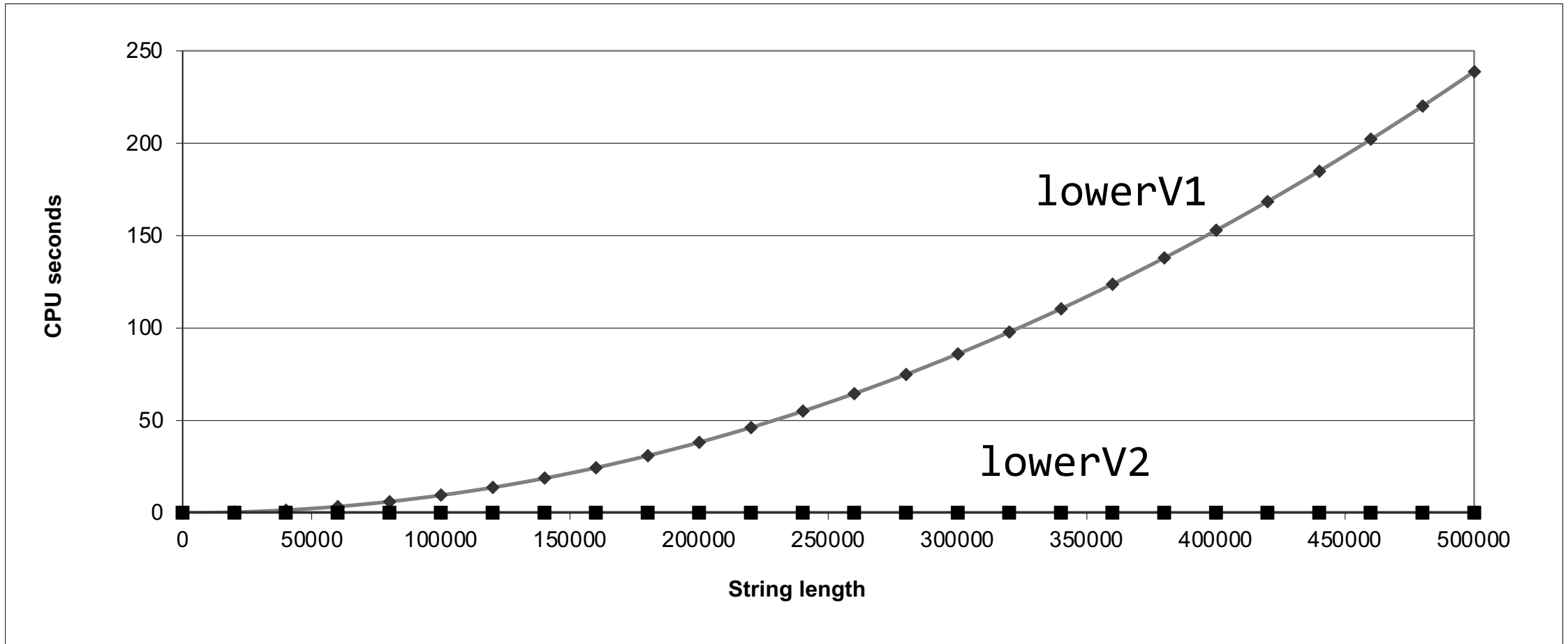
Improving Performance

- Move call to **strlen** outside of loop
 - Note that **you** (yet not the compiler) know that the length `len` does not change from one iteration to another
 - You took it upon yourself to perform “code motion”

```
void lowerV2(char *s)
{
    size_t i;
    size_t len = strlen(s);
    for (i = 0; i < len; i++)
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] -= ('A' - 'a');
}
```

Lower Case Conversion Performance: lowerV2

- Time doubles when double string length – Linear performance of lowerV2



Optimization Blocker: Procedure Calls

- Why couldn't compiler move `strlen` out of inner loop?
 - A function might be stateful 
 - e.g., alters global state each time called
 - Function might not return same value for given arguments
 - e.g., depends on global state (for instance, in multi-core parallelism)
- The takeaway message
 - Compiler must treat procedure call as a black box
 - Weak optimizations near/around procedure call (stumbling block)
 - Compiler cannot read your mind; it does exactly what you told it to do
- Possible remedies (since you know what goes on, unlike the compiler):
 - Try to do your own code motion (you should know if it's safe or not)
 - C++: Use of inline functions
 - GCC does this with `-O1`
 - Within single file
 - Use LTO (as we discussed)

This, and nothing more, is the content of a file in which your `strlen` function is defined



```
static size_t lencnt = 0;

size_t strlen(const char *s)
{
    size_t length = 0;
    while (*s != '\0') {
        s++; length++;
    }
    lencnt += length;
    return length;
}
```

In this example, each time this `strlen` is called a variable is incremented (cooked up, but makes a point)

Memory matters, and the trickiness of pointers

```
// sum entries in each row of A and store in vector b
void sum_rows1(double *A, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        b[i] = 0;
        for (j = 0; j < n; j++)
            b[i] += A[i*n + j];
    }
}
```



Assembly code, sum_rows1, inner loop (after j)
(gcc, optimized w/ -O1, from <https://godbolt.org/>)

.L3:

movsd	xmm0, QWORD PTR [rdx]	Bring b[i] from memory into register xmm0
addsd	xmm0, QWORD PTR [rax]	Add to register xmm0 the right A entry from memory
movsd	QWORD PTR [rdx], xmm0	Move updated b[i] value back into memory
add	rax, 8	Make rax have the address to next entry in A (double takes 8 bytes)
cmp	rax, rcx	Compare i with n (latter stored in rcx)
jne	.L3	Jump to .L3 if not equal

- Code updates `b[i]` *in memory* on every iteration
- Why couldn't compiler optimize this away?

Memory Aliasing

```
/* Sum rows of n X n matrix A and store in vector b */
void sum_rows1(double *A, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        b[i] = 0;
        for (j = 0; j < n; j++)
            b[i] += A[i * n + j];
    }
}
```

```
int main() {
    double A[9] = {0, 1, 2, 4, 8, 16, 32, 64, 128};

    double *b;
    b = A + 3;
    sum_rows1(A, b, 3);
    return 0;
}
```

```
double A[9] =
{ 0,  1,  2,
  3, 22, 224},
 32, 64, 128};
```

Value of b:

init: [4, 8, 16]

i = 0: [3, 8, 16]

i = 1: [3, 22, 16]

i = 2: [3, 22, 224]

A,9	0x000000ff3a8ffc60	double[9]
[0]		0 double
[1]		1 double
[2]		2 double
[3]		3 double
[4]		22 double
[5]		224 double
[6]		32 double
[7]		64 double
[8]		128 double
b,3	0x000000ff3a8ffc78	double[3]
[0]		3 double
[1]		22 double
[2]		224 double

- The code updates **b[i]** on every iteration. It does so because:
 - Must consider possibility that these updates will affect program behavior
 - Value of “b” might change from when i=3 to when i=4 (parallel thread might interfere)

Removing Aliasing

- Assume that there is no aliasing
 - That is, no overlap in calculations
- You can do code motion
 - Result: Accessing array b only once!!!
- Number of instructions drops down
 - Performance goes up

```
/* Sum rows of n X n matrix A and store in vector b */  
void sum_rows2(double *A, double *b, long n) {  
    long i, j;  
    for (i = 0; i < n; i++) {  
        double val = 0;  
        for (j = 0; j < n; j++)  
            val += A[i*n + j];  
        b[i] = val;  
    }  
}
```



Assembly code, sum_rows2, inner loop (after j)
(gcc, optimized w/ -O2, from <https://godbolt.org/>)

.L3:

```
    addsd    xmm0, QWORD PTR [rax]  
    add      rax, 8  
    cmp      rax, rcx  
    jne      .L3
```

Bring A entry from memory and add to what's in xmm0

Make rax point to next entry in row of A

Have you exhausted all entries in row "i" of A?

Jump back and do another trip if not more elems in row "i"

[Next thing to consider:] Memory Matters

.L3:

```
movsd    xmm0, QWORD PTR [rdx]
addsd    xmm0, QWORD PTR [rax]
movsd    QWORD PTR [rdx], xmm0
add      rax, 8
cmp      rax, rcx
jne      .L3
```

.L3:

```
addsd    xmm0, QWORD PTR [rax]
add      rax, 8
cmp      rax, rcx
jne      .L3
```

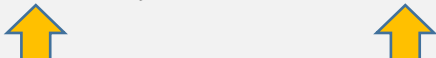
- Bottom line: → **QWORD PTR** shows up: three times (left) vs. one time (right)
 - That is, three vs. one memory accesses. Leads to **significant speedup**

Optimization Blocker: Memory Aliasing

- Two different memory references specify single location

- Easy to have happen in C - you have low level access to memory through pointers

```
#include <stdlib.h>
/* Sum rows of n X n matrix A and store in vector b */
void sum_rows1(double * restrict A, double * restrict b, long n)
{
    long i, j;
    for (i = 0; i < n; i++) {
        b[i] = 0;
        for (j = 0; j < n; j++)
            b[i] += A[i * n + j];
    }
}
```



- Way around it:
 - Used local variables, like in the previous example – handy to accumulate within loops
 - You'll help the compiler – it can issue better instructions
 - Use the **restrict** keyword, see code snippet
 - A type qualifier that can be applied to pointers to provide a strong hint to the compiler that for the lifetime of the pointer, no other pointer will be used to access the same object.

Quiz: What does the restrict keyword signify in this context?

```
void func(int * restrict x, int * restrict y) {  
    *x = 10;  
    *y = 20;  
}
```

- a) It guarantees that the values pointed to by x and y are different.
- b) It prevents modification of the pointers x and y within the function.
- c) It hints to the compiler that x and y likely point to overlapping memory regions.
- d) It provides a strong hint to the compiler that x and y point to exclusive memory regions.

Vector Instructions

- Basic idea: Use fat registers to perform the same operation on all variables stored in register
- X86 has, for instance, 512 bit-wide registers
 - Technology called AVX512 (Advanced Vector EXtension)
- Since chip in this example is Haswell, it only has 256-bit registers
 - Technology called AVX2

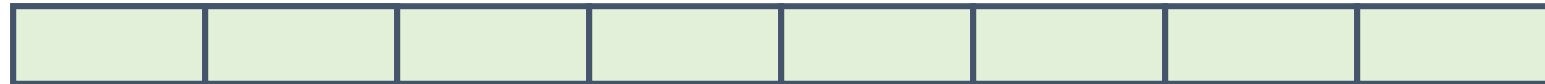
Programming with 256 Bit-wide AVX2

- What can you do with AVX2?

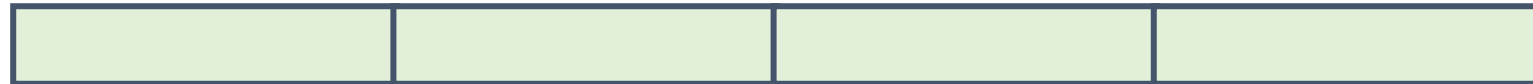
- Store 8 32-bit integers:



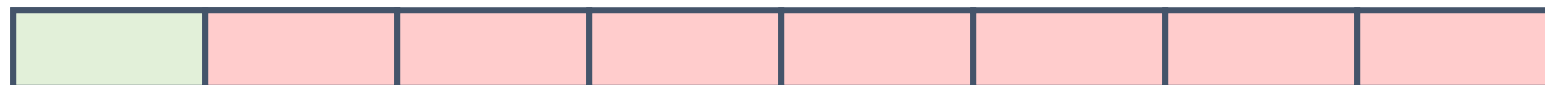
- Store 8 single-precision floats:



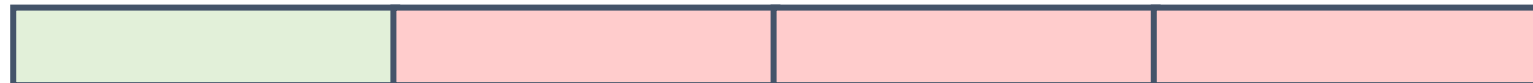
- Store 4 double-precision floats:



- Store 1 single-precision float but then waste bits:



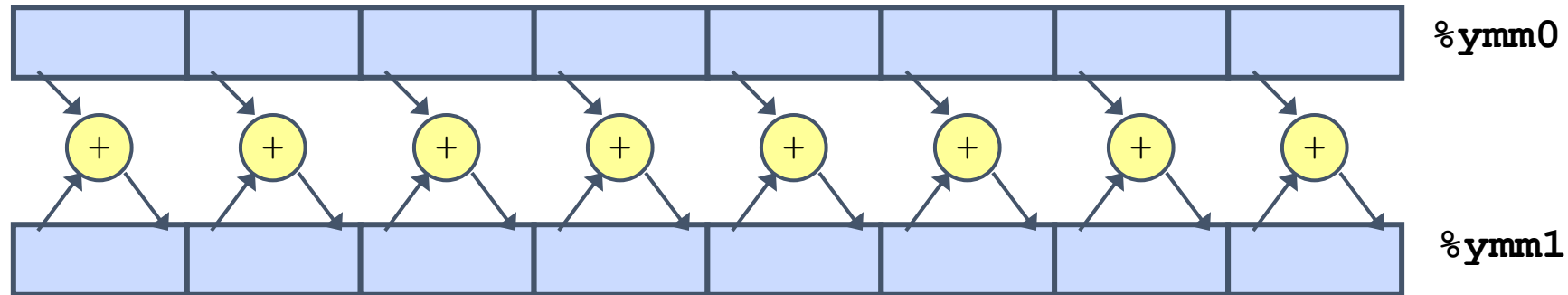
- Store 1 double-precision float but then waste bits:



SIMD Operations, with AVX2

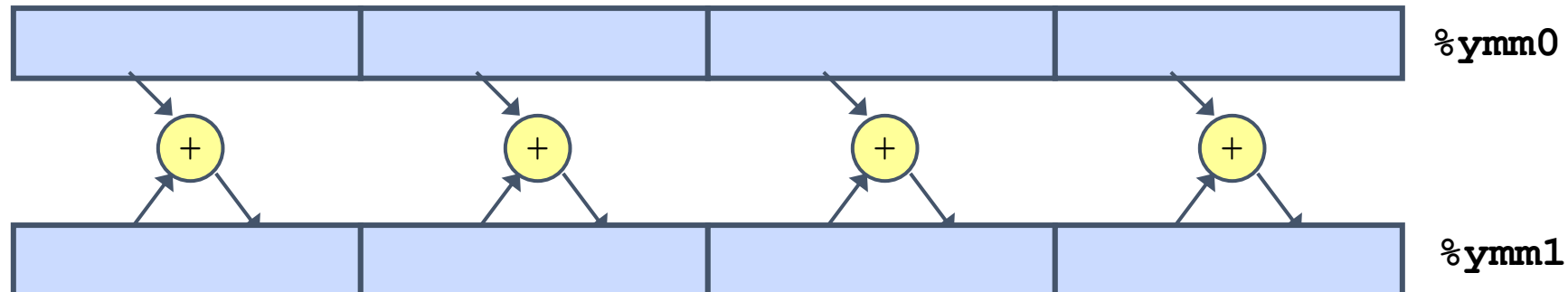
■ SIMD Operations: Single Precision (float)

vaddsd %ymm0, %ymm1, %ymm1



■ SIMD Operations: Double Precision (double)

vaddpd %ymm0, %ymm1, %ymm1



SIMD Example: Add Two Vectors

- **#include <immintrin.h>**: Includes the header file for SSE intrinsics.
- **__m128**: Data type representing a 128-bit vector of four single-precision floats.
- **_mm_loadu_ps**: Loads four floats from unaligned memory into a __m128 vector.
- **_mm_add_ps**: Adds two __m128 vectors element-wise.
- **_mm_storeu_ps**: Stores four floats from a __m128 vector to unaligned memory.

```
#include <immintrin.h>

void vector_add(float *a, float *b, float *c, int n) {
    // Assuming n is a multiple of 4 for simplicity
    for (int i = 0; i < n; i += 4) {
        __m128 a_vec = _mm_loadu_ps(&a[i]);
        __m128 b_vec = _mm_loadu_ps(&b[i]);
        __m128 c_vec = _mm_add_ps(a_vec, b_vec);
        _mm_storeu_ps(&c[i], c_vec);
    }
}
```

SIMD Example: Dot Product

- **Initialize accumulator:** The acc variable is initialized to zero using `_mm_setzero_ps`.
- **Load vectors:** Load four elements from each vector into `a_vec` and `b_vec`.
- **Element-wise multiplication:** Multiply corresponding elements of `a_vec` and `b_vec` using `_mm_mul_ps`.
- **Accumulate results:** Add the result of the multiplication to the accumulator using `_mm_add_ps`.
- **Horizontal sum:** Use `_mm_hadd_ps` twice to reduce the four elements in `acc` to a single float.
- **Extract result:** Extract the final result using `_mm_cvtss_f32`.

```
#include <immintrin.h>
```

```
float dot_product(float *a, float *b, int n) {  
    // Assuming n is a multiple of 4 for simplicity  
    __m128 acc = _mm_setzero_ps();  
    for (int i = 0; i < n; i += 4) {  
        __m128 a_vec = _mm_loadu_ps(&a[i]);  
        __m128 b_vec = _mm_loadu_ps(&b[i]);  
        acc = _mm_add_ps(acc, _mm_mul_ps(a_vec, b_vec));  
    }  
    →
```

```
    ←  
        // Horizontal sum of the accumulator  
        acc = _mm_hadd_ps(acc, acc);  
        acc = _mm_hadd_ps(acc, acc);  
        return _mm_cvtss_f32(acc);  
    }
```

Library for SIMD programming to make your life easier

- SIMDe: <https://github.com/simd-everywhere/simde>

SIMD Everywhere

all contributors 78

chat on gitter  codecov 79%

The SIMDe header-only library provides fast, portable implementations of [SIMD intrinsics](#) on hardware which doesn't natively support them, such as calling [SSE](#) functions on ARM. There is no performance penalty if the hardware supports the native implementation (e.g., SSE/[AVX](#) runs at full speed on [x86](#), [NEON](#) on [ARM](#), etc.).

Quiz: Loop Unrolling

Modify the following code to a 4-way unrolled version of the loop:

```
#pragma omp parallel for schedule(static)
for(int i = 0; i < N; i++) {
    result[i] = some_computation(data[i]);
}
```